📖 TTSWAP 去中心化商场白皮书

> **一句话介绍**：TTSWAP 是一间开在区块链上的、永不打烊、无老板、无收银员的**去中心化商场**。
> 任何人都可以上架商品、摆货架、当顾客、当供货商。
> 全场共用同一个**中央收银台**，任意两条货架之间都能直接换货；供货商存货即赚手续费，承诺型货架还可同步领 TTS；交易费用在链上按规则自动分给"供货商 / 品牌方 / 门店 / 推荐人 / 顾客 / 商场"六方。

> **本白皮书写给谁看**：想理解 TTSWAP 的非技术读者（用户 / 运营 / 商家 / 投资人）。
> 全文以"商场故事"串起所有机制，所有链上概念都用日常语言重讲一遍。

目录

1. 引子：一间不打烊的去中心化商场
2. 商场常识：商品、货架、供货凭证、硬通货
3. 中央收银台：一个货架可以换任意另一个货架
4. 商品上架：开新货架与品牌方权益
5. 供货商补货：为什么只用一种商品也能补
6. 顾客买单：两种买东西的方式
7. 商家与远程支付（X402）
8. 货架库存的"安全带"
9. 供货商结账：怎么取回本金与利润
10. 商场六方分账：每一笔交易被谁分了
11. 管理层：谁可以改价、谁可以关店
12. TTS 代币：商场最稀缺的"股权凭证"
13. 升级、熔断与永久不可变
14. 典型场景速查
15. 联系方式 & 法律许可
16. 附录：机制速查、参数极限、与主流 DeFi 的对照

1 引子：一间不打烊的去中心化商场

想象一间开在云端的商场：

- 没有老板，没有收银员，但有一部不会出错的电脑（= 智能合约）；
- 没有营业时间，全年 365 天 24 小时开门；
- 每个商品都有一个**货架**，顾客可以任意在两条货架之间交换商品；
- 任何人都可以上架一个新商品，也可以给已有货架补货,得到收益；
- 每次交易的手续费由电脑按规则自动分给"供货商 / 品牌方 / 门店 / 推荐人 / 顾客 / 商场"六方；
- 每个拥有TTS代币的用户都是商场的老板：享受监督,提案,投票,分红的权利；

TTSWAP 就是这间商场的链上版本。整本白皮书都在讲这间商场怎么运作。

1.1 为什么要开这么一间商场

传统加密世界里，"想把商品 A 换成商品 B"通常要走进一家"**只卖 A 和 B 两种商品的专营小店**"——在 Uniswap 这类 DEX 里，这种小店叫 **pair**（交易对）。如果街上没人开过这家小店，就得"A → 美元 → B"换两家店走两趟。

直觉对照：

- **传统 DEX 像一条商业街**：每个 pair 都是一家**只卖 2 件商品的专营小店**，自己备货、自己算账、自己定价；
- **TTSWAP 是商业街尽头的一间大型商场**：全场只有 **一个中央收银台**，每种商品一条货架，任意两条货架直接换货。

一条商业街的问题：

- **商品种类越多，小店越多，备货越分散**。100 种商品理论上要开 **4,950 家**只卖 2 件商品的小店；一种商品要放到100个小店中去；
- **开店门槛高**：小店上架的两件商品必须**等价值备货**（例如 1 ETH + 2000 USDT），手头只有其中一种、或者两种价值不匹配的供货商**根本没办法开店**；
- 新商品没人给它开小店，等于**无法交易**；
- 顾客要自己研究"先进哪家店、再进哪家店"，中间滑点、Gas、时间都要付两次；
- 供货商把货放进某一家小店，就永远只能服务这家店的顾客，**街上其它小店想用也用不了**。

TTSWAP 的做法很简单：**把所有商品搬进同一间商场、共用同一个中央收银台**。于是：

- 100 种商品就是 100 条货架，不再是 4,950 家小店；
- 新商品只要挂上货架，立刻可以被所有顾客兑换；
- 供货商往一条货架补货，**整间商场的流动性深度都会被这条货架用到**；
- 顾客走进商场只做一件事："**我想把这个商品换成那个商品**"，收银台一次结算搞定。

1.2 本白皮书讲什么

> 这间商场的 5 条主线：
>
> 1. **商品怎么上架**（谁能新开货架 / 怎么定初始价）
> 2. **谁给货架补货，补货怎么赚钱**（供货商机制 + 无常损失防御 + 撤货规则）
> 3. **顾客怎么买东西**（两种下单方式 + 远程支付 + 防夹攻击）
> 4. **商场怎么让所有人都能赚钱**（六方分账：供货商 / 品牌方 / 门店 / 推荐人 / 顾客 / 商场）
> 5. **TTS 代币怎么分**（4C 股权 + 翻倍解锁 + 供货即挖矿 + 回购销毁）

读完这五条主线，就算你没写过一行 Solidity，也能理解 TTSWAP 里每一个关键机制**在商场世界里对应的动作**。

2 商场常识：商品、货架、供货凭证、硬通货

在正式进入交易、补货之前，先把商场里的五个基本概念讲清楚。

2.1 货架（池）：商品的唯一"家"

商场里每种商品对应**唯一一条货架**，货架上记录四项信息：

货架四要素	商场叫法	对应链上字段
市场价值 V	这货架商品在这家商场里价值多少	`investState.amount1()`
当前库存 Q	货架上现在有多少件商品	`currentState.amount1()`
累计进货 I	历史所有供货商累计进了多少件	`currentState.amount0()`
总供货份数 S	所有供货商合起来持有的"货权总份数"	`investState.amount0()`
总虚拟数量 R	所有供货商虚拟的总数量	`goodconfig.amount1()`

直觉上：

- **V 是每种商品的"标价"，不会因为进货出货直接改变**；
- **Q 是当前货架上的商品数量**（会随买卖增减）；
- **I 是这条货架上有多少货是"真的被供货商放进来的"**（永远记账，反映支撑深度）；
- **S 是所有供货商持有的货权总份数**（撤货时按这个比例分钱）。
- **R 是这货架上不能交易的总数量**

规则 1：同一种商品全商场只有一条货架（合约层强制： `goods[goodid].owner != 0` 则不可重复开）。

规则 2：每种商品用 `T_GoodKey` 三元组标识——（资产类型，合约地址，子编号）。普通 ERC20 代币的子编号固定为 0；未来股票化代币（ERC-1155）、预测市场份额（ERC-6909）可用不同子编号区分同一合约下的多种标的。

2.2 供货凭证（Proof of Investment）

每次有人往某条货架补货，商场会发一张**供货凭证**，上面写：

```
供货凭证 {
    我给哪条货架供了货（currentgood）
    我占这条货架多少份货权（shares）
    我当时把货折算成了多少价值（state）
    我送了多少件虚拟陈列量（invest 虚拟）
    我实际交付了多少件（invest 实际）
}
```

关键点：**凭证不是一张可以二次交易的 NFT，它是一个按"身份 + 货架"唯一绑定的账本条目**。

> `proofid = keccak256(owner, goodid)` —— 为**单币凭证**，不再绑定"配对价值币"。

同一个人给同一条货架补多次，会累加到**同一张凭证**上（不会变成 N 张）。换言之：**身份和货架配对，只有一张凭证**。

2.3 商品的两种等级

v2 简化了商品分级：**不再区分"元商品"与"硬通货"两条独立建池路径**，统一用 `initGood` 开货架，再由管理员提权。

等级	谁来担任	特权	商场里的直觉
硬通货（Value Good）	市场认可度高的商品，由 **CEO（Market Admin）** 按社区决议提权	可作价值锚定； `safeLine` 默认可放宽	商场里的**"畅销一级商品"**（USDT / ETH / WBTC 等，顾客愿意当货币用）
一般商品（Normal Good）	任何新上架的商品默认是这个等级	受**安全线（safeLine）**约束：大额买卖不能瞬时把货架推离健康区间	商场里的**长尾商品**：想无限往里堆货、再一把套走硬通货，会被安全带拦住

为什么一般商品要做安全约束？如果商场来者不拒，就会出现"供货商拼命塞垃圾货，把商场里真正的 USDT / ETH 套走"。协议用 `safeLine` 上下界（在合约层设安全带，详见第 8 节。

2.4 硬通货为什么这么特别

硬通货是商场执行层（CEO，持 Market Admin 钥匙）挑出来的"市场广泛认可、价值相对稳定"的商品。被标为硬通货后：

- 任何人补货都按**池内现价**计；
- 可作定价锚，且 `safeLine` 通常配置更宽松；
- 手续费六方分佣在硬通货货架上更稳定。

一个新商品要升格为硬通货，必须由 **CEO（Market Admin）** 按社区 II 类提案通过后，调用 `modifyGoodByAdmin` 打开硬通货标志位——**不是自动的，要经过提案审批**。

2.5 商场里的"中央收银台"：Singleton 合约

整间商场只有**一个中央收银台**，所有货架都接到这台收银机上：

- 前端对接 **Proxy 合约地址**（ `TTSwap_Market_Proxy` ），逻辑可升级、状态不变；
- 所有货架挂在同一个 `mapping(goodid => S_GoodState)` 下， `goodid` 由 `T_GoodKey` 算出；

- 所有交易、补货、撤货都在这个合约里完成；还支持 `multicall` 一次交易批量办多件事。

好处：

- 流动性深度自然聚合（见第 3 节）；
- Gas 消耗低（一次调用就结束，详见附录）；
- 安全审计面小（逻辑集中，没有跨合约不一致）。

3 中央收银台：一个货架可以换任意另一个货架

3.1 传统 DEX 的痛点：一条商业街上 N 家专营小店

以 Uniswap 为例：想把面包换成牛奶，必须先有人在链上开一家"**面包 ↔ 牛奶专营店**"（pair）。这家店**只卖面包和牛奶两种商品**，不卖第三种。如果这家店没人开过，就得"面包 → 美元店 → 牛奶店"走两家店、换两次。

商品数量	需要开多少家"只卖 2 件商品的小店"
3 种	3 家
10 种	45 家
100 种	4,950 家
N 种	N(N-1)/2 家

问题：**每家小店备货独立、商品深度差**，长尾商品根本没人愿意开店。

3.2 TTSWAP 的做法：一间商场，一个中央收银台

TTSWAP 把整条商业街塞进一间大型商场，全场共用**同一个中央收银台**。当你想把商品 A 换成商品 B，商场的做法是：

第 1 步：收银台先给你的 A 商品定个"购买力"

- 看一眼 A 货架：当前货架上有多少件 A（Q_A），A 总价值多少（V_A）；
- 按公式算出你带来的这批 A 能换算成多少"商场内部购买力"（记作 ΔV）。

第 2 步：再用这笔购买力去换 B

- 看一眼 B 货架：当前货架上有多少件 B（Q_B），B 总价值多少（V_B）；
- 按公式算出你能拿走多少件 B（记作 Δb）。

整个过程是**一次合约调用**完成的，你从头到尾只做一个动作："把 A 换成 B"。

两条底层公式（给想看数学的读者）

第 1 步和第 2 步其实就是链上代码里的两条代数式。

公式 1：数量 → 价值（你给 A 货架送进 Δa 件商品，换算出"商场内部购买力" ΔV）

$$\Delta V = \frac{2 \cdot V_A \cdot \Delta a}{2 \cdot Q_A + \Delta a}$$

公式 2：价值 → 数量（用购买力 ΔV 去 B 货架换走多少件 B）

$$\Delta b = \frac{2 \cdot Q_B \cdot \Delta V}{2 \cdot V_B + \Delta V}$$

两条公式合起来满足三件事：

- **价值守恒**：你送进去 A 的总价值 ΔV，严格等于你拿走 B 的总价值；
- **滑点有界**：送的越多，单价越差，不会出现"巨鲸一次扫空"的情形；
- **O(1) 常数开销**：全程只有加减乘除，没有指数、没有迭代、没有外部调用，所以 Gas 极低（见附录的 Gas 对照）。

> 更完整的推导（CPMM / Weighted / StableSwap 三重等价性证明）请移步 白皮书技术版 附录 A–D。

3.3 两个非常关键的好处

好处 1：任意两商品直达

无论你想"面包换牛奶"、"鸡蛋换黄豆"、"洗发水换螺丝刀"，对商场来说都是同样的两步动作——**不需要一条商业街上开 4,950 家小店，一个中央收银台就能处理所有 4,950 种兑换需求**。

好处 2：流动性自然聚合

供货商给"面包"货架补货，这批面包的"购买力"可以被**所有想把 XX 换成面包的顾客**共享——不会像 Uniswap 那样只服务于"面包 ↔ 牛奶"一家专营小店。

对照	传统 DEX（商业街 · N 家专营小店）	TTSWAP（一间商场 · 一个收银台）
新商品上架	必须有人开一家"X ↔ 新商品"小店，长尾商品无人问津	挂上货架就能被所有人兑换
流动性分布	切分到各家小店，各自为战	全场共享一个收银台，深度天然聚合
用户路由	找路径、跨合约、算滑点、多次手续费	永远直接交易
新商品对老用户	无影响（老小店不知道它存在）	自动加入结算网络

3.4 收银台背后的数学（直觉版）

第 1 步和第 2 步其实是两条简单的代数公式，公式的特点是：

- **数值对等**：你送进来 A 的总价值，永远等于你拿走 B 的总价值；
- **有滑点**：送得越多，单位商品换到的 B 越少（防止巨鲸一扫而空）；
- **无指数、无迭代**：只有加减乘除，所以 Gas 极低。

如果你想看完整公式，请去白皮书（技术版）附录 A。本白皮书保证：**你不看公式也能理解商场怎么运作**。

4 商品上架：开新货架与品牌方权益

4.1 两种核心动作

商场的每种商品只能挂一次货架。任何人参与流动性，只有**两条路**：

方式	谁用	场景
`initGood` （开新货架 + 自定初始价）	任何人	第一次为某种 Token 开货架
`investGood` （单币补货）	任何人	给已存在的货架追加存货

4.2 开新货架： `initGood`

要开一个全新货架，你需要告诉商场三件事：

1. **你要上架的商品是什么**（ `T_GoodKey` ： 类型 + 合约地址 + 子编号）；
2. **你打算放多少件**（ `amount1` = 初始库存 Q_0）；
3. **你给它标一个初始总价值**（ `amount0` = 初始价值 V_0，即定价锚）。

合约会做 4 项检查：

- $V_0 \geq 5 \times 10^{14}$（约 500 USDT 等价量级，太便宜禁止开店， `TTSwapError(35)` ）；
- $V_0 \leq 2^{109}$（防溢出）；
- $500{,}000 \leq Q_0 \leq 2^{109}$（库存不能是尘埃池， `TTSwapError(36)` ）；
- 这个商品之前**没人开过货架**（ `TTSwapError(5)` ）。

通过后，你就成了这条货架的**品牌方（Owner）**：

- 自动获得一张**供货凭证**（份额 = 初始存货量）；
- **不会自动挖 TTS**——开货架本身不质押；只有后续 `investGood` 且货架标记为"承诺型（isPromised）"时才记 TTS 算力；
- 所有顾客立刻可以通过收银台用 USDT / ETH 等硬通货来买你的新商品。

举例：一个新商品上架的完整流程

设 Alice 要发行一个新品牌叫 `$NEW`：

- 她铸了 100 枚 `$NEW`；
- 她认为公允价格大约是 1 USDT / 枚（即 100 USDT 等价价值）；
- 她调用 `initGood(NEW, packed(V₀=100 USDT 等价, Q₀=100 枚), ...)` 。

完成后：

- `$NEW` 货架诞生： `V = 100 USDT` ， `Q = 100 枚` ， `I = 100 枚` ；
- Alice 持有初始供货凭证： `shares = 100` ；
- 任何持有 USDT 的顾客可以调用 `buyGood(USDT, NEW, ...)` 直接换 `$NEW` 。

关键：Alice **只准备了 100 枚 `$NEW`** ，没有准备任何 USDT 对手资产。

第一个给某商品开货架的权益（首发运营权）

TTSWAP 把"第一个在商场里为某种 ERC20 开货架"这件事当成**一次商业拓荒**——它意味着货架搭建、社区冷启动、品牌方关系维系、早期买家教育都由第一位开店人承担。为了激励这项工作，协议给首发者一份**有期限的运营权**：

- **运营期**：生态建议 **默认 2 年**（合约不硬编码到期，靠 `changeGoodOwner` 市场化移交）；
- **运营收益**：期间这条货架上撤资分润里的**操作人分佣（operatorFee）**归品牌方所有，典型 **1%–3%**；
- **到账方式**：佣金累积在 `commission[owner]` ，随时 `collectCommission` 领取；
- **权限延伸**：品牌方可用 `modifyGoodByGoodOwner` 调整费率、放大倍数、撤货切片；可用 `lockGood` 紧急冻结。

设计动机：让第一个开店的人在前 2 年里拿到明确、可预期的经济回报，避免"为他人作嫁衣"。2 年到期后，品牌方身份可以按 §4.5 描述的方式通过**运营权拍卖**过渡给更有能力的继任者，让货架在生命周期里始终由"最有动力运营它的人"来运营,可以自己参与拍卖,获得运营权。

4.3 定价锚商品：谁来做"商场里的美元"

不再单独设"元商品"建池入口。实践中，**第一个被社区提权为硬通货的稳定币（通常是 USDT）**自然承担定价锚角色：

- 新货架开池时，创业者自定 V_0/Q_0 即初始价；
- 后续交易、补货、TTS 解锁判价等，都通过 `ishigher(TTS, USDT, ...)` 等路径引用硬通货货架现价；
- 硬通货由 **CEO（Market Admin）** 按社区决议 `modifyGoodByAdmin` 提权，不是自动晋级。

> **实务**：主网部署通常以 USDT 为首要硬通货，但协议不与某一稳定币绑定——社区可用 II 类提案提权 USDC、DAI 等。

4.4 硬通货（Value Good）的晋升路径

其它商品如果要晋升为硬通货，流程是：

1. 先用 `initGood` 正常开货架（默认一般商品）；
2. 积累声誉与深度后，**CEO（Market Admin）** 按社区 II 类提案调用 `modifyGoodByAdmin` 打开硬通货标志；
3. 之后这个商品就：

- 可以作为其他新商品的"定价锚"（虽然一般商品也可以但硬通货更权威）；
- 享受"不受库存硬约束"的待遇；
- 配置的分佣比例、放大倍数通常会被调整到更优值。

这是一个"人治 + 声誉"的过程，不是自动晋级——这样设计是为了防止刚上线的商品冒充"硬通货"骗取流动性。

4.5 升级 / 放弃品牌方身份

品牌方也可以随时把货架的品牌方身份让渡给别人：

- `changeGoodOwner` （仅管理员权限）：把品牌方身份迁移到另一个地址；
- `lockGood` （品牌方或管理员权限）：紧急冻结这条货架的交易。

注意：品牌方的身份和供货凭证（shares）是**解耦**的——让渡品牌方身份不会转移你持有的供货凭证。

运营权拍卖：货架转手的市场化路径

§4.2 提到的**2 年首发运营期**到期后（或者现任品牌方自愿提前退出），货架的**运营权**可以通过拍卖的方式移交给下一位运营者，让货架在整个生命周期里都归"出价最高、最愿意为它投入"的人打理。

为什么用拍卖而不是直接转让?

因为货架的运营权不是一份静态的头衔，而是**未来交易手续费分佣的折现流**。不同候选人对同一条货架的估值可能差距很大（取决于他们能带来的社区、渠道、营销能力），市场拍卖是最公平的定价方式。

拍卖流程与资金分配

1. 拍卖过程在链下 / 专用前端完成（本白皮书不强制实现方式，社区可以选择英式拍卖、荷式拍卖或密封报价），竞拍标的就是这条货架的**品牌方身份**；
2. 最终拍卖款按协议规则**自动分成两笔**：
   - **70% 归原运营者**（即现任品牌方）——奖励他在前期建设、社区运营、早期买家教育上的投入；
   - **30% 归商场协议**（汇入 `commission[address(0)]` 社区金库）——用于后续的回购销毁 / 运营预算 / 生态激励（具体用途由社区 III 类提案决定）；
3. 分账确认 + 款项到账后，由**管理员（Market Manager）**发起 `changeGoodOwner(goodid, 新品牌方地址)` 把货架身份迁移过去；
4. 新品牌方从迁移完成那一刻开始享有 `operatorFee` 分佣，并重新获得 `modifyGoodByGoodOwner` / `lockGood` 等品牌方权限。

边界与保护

- 拍卖**只移交运营权（owner 身份）**，不会触碰任何供货商的供货凭证（ `proofs` ）或 TTS 算力；
- 如果拍卖失败，由原运营者继续运营（运营权不会被强制剥夺）；
- `changeGoodOwner` 由管理员权限执行，意味着每次移交都会进入链上事件日志，便于社区审计；
- 拍卖款的 70 / 30 分配是**协议层建议**，具体实现（托管合约、保证金、异议期等）会随社区治理迭代。

直觉：就像一家线下商铺的经营权到期后，房东会通过公开招租让下一位店主接手。TTSWAP 把这件事搬到链上，让货架的运营权能够在市场里自由流转，同时保障原运营者的应得回报、并把协议自身的利益与货架长期表现绑定。

5 供货商补货：为什么只用一种商品也能补

5.1 只用一种商品补货，这合理吗?

在传统 DEX 里，补货往往要同时准备两种等值商品。TTSWAP 更贴近真实商场：**只带一种商品来，也能补货**。

调用 `investGood(goodKey, packed(0, quantity), ...)` 即可—— **一律按货架当前价自动计价**。

5.2 单币补货规则

场景	调用方	说明
按货架当前价补货	任何用户	协议按 `V/Q` 折算你这批货的价值；若申报价高于池价会 revert（`TTSwapError(47)`）
最小价值	任何用户	单次补货折算价值须 ≥ 10^{12}（`TTSwapError(38)`）

5.3 单币补货的 6 步流程

以 Bob 给 `$NEW` 货架补货为例：

1. **收银台先看货架状态**：读取当前 V、Q、I；
2. **按当前价折算**：扣除投资费后，按池价计算本次贡献价值；
3. **读取放大倍数**（`enpower`）：按池深度动态缩放，上限由 `power` 配置；
4. **虚拟放大**：真实进货按倍数记入虚拟陈列深度；
5. **计算供货凭证份额**：按份额比例给 Bob 增发凭证；
6. **写回货架 + 条件性 TTS 记账**：仅当货架标记 `isPromised` 时， `stake` 记入 TTS 算力。

价格中性：补货前后，池内单位定价由公式保证不会因单笔补货剧烈偏移。

5.4 为什么"单币补货"不会被套走其它商品

这是很多读者最关心的问题：**如果只用一种商品就能补货，套利者不是可以"拼命补一般商品然后套走 USDT"吗?**

答案是：**不能轻易做到**。协议用 **安全线（safeLine）** 限制单笔交易对货架深度的冲击——当卖压把库存推到上界，或买盘把库存抽到下界，收银台会拒收（`TTSwapError(55/56)`）。

5.5 一个直觉极强的现实对照

> 一个供货商给社区商场送 1000 瓶矿泉水，实际送了200瓶水，虚拟陈列了800瓶水，进货价 10 元/瓶。
> 那这个货架放置了最多放 1000 瓶(实际+虚拟陈列)。
> 如果没人来买，其它用户希望卖出500 瓶换其它商品，商场老板一定会拒收——**货架满了，再收下就得烂在仓库，挤占其他商品的位置**。
> 只有当消费者真的开始买水，货架腾出 200 个空位，其它用户才可能卖出200个商品来换其它商品。

TTSWAP 对一般商品的处理和这个商场完全同构：

现实商场	TTSWAP
货架容量 1000 瓶	I + 虚拟陈列量（LP 支撑上限）
供货商来补货	顾客卖一般商品给货架（`buyGood A → B`）
消费者来买水	顾客买一般商品（`buyGood B → A`）
老板拒收超量	`TTSwapError(55)` / `TTSwapError(56)`（安全线）
货架上的牛奶、饼干	商场里的其它商品（USDT、ETH 等）

6 顾客买单：两种买东西的方式

顾客在 TTSWAP 商场有两种下单方式：

方式	叫法	场景
`buyGood`（指定"付出多少"）	Exact-In	"我有 100 块钱，能买多少面包？"

方式	叫法	场景
`payGood`（指定"拿回多少"）	Exact-Out	"我要恰好买 10 个面包，请告诉我要付多少钱"

两种下单方式都支持：

- **任意两商品直达**（商品 A → B，不需要为它们单独开一家 pair 小店）；
- **链上 deadline 保护**（过期自动作废，防陈旧签名）；
- **最小 / 最大限额**（防滑点滥用）；
- **商家模式**（见第 7 节 X402）。

6.1 buyGood ： 我有 X，求 Y

叙事：你拿着 100 USDT 去商场，想换成 ETH。你告诉收银台：

> "我给你 100 USDT，ETH 给我多少都行（但不能少于 0.025 ETH，否则就别给了）。"

合约执行：

1. 收银台用 100 USDT 算出购买力 ΔV；
2. 用 ΔV 去 ETH 货架，算出给你多少 ETH；
3. 验证 $\geq$ 你的最小下限（防滑点），不满足就回滚；
4. 从你账户扣 100 USDT，转给你对应的 ETH；
5. 扣一小笔交易费，自动记到六方分账的池子里（见第 10 节）。

关键字段：

- `amount0` = 你付出的 USDT 数量 = 100（固定）；
- `amount1` = 你接受的 ETH 最小值 = 0.025（滑点下限）；
- `deadline` = 截止时间戳（强烈建议非零）。

6.2 payGood ： 我要精确拿回 Y

叙事：你要恰好买 10 个面包送礼，多一个少一个都不行。你告诉收银台：

> "给我恰好 10 个面包，我愿意付不超过 50 USDT。"

合约执行：

1. 收银台反向算：10 个面包需要多少 ΔV？
2. 反向算：要补这么多 ΔV，你得付多少 USDT?
3. 如果 USDT 数量超过你设的上限（50），就回滚；
4. 从你账户扣对应 USDT，转给你恰好 10 个面包。

关键字段：

- `amount1` = 精确想拿到的面包数 = 10（固定）；
- `amount0` = 你愿意付出的 USDT 最大值 = 50（滑点上限）；
- `external_info.amount0` = 可以塞业务信息（如支付订单号）；
- `external_info.amount1` = 截止时间戳（强烈建议非零）。

6.3 payGood 的"同币种快路径"：商家收款专用

有个很常见的场景：

> **小明要用 USDT 付给小店 10 USDT 的订单，小店支持 TTSWAP 收款。**

这根本不是"兑换"，是**纯转账**。TTSWAP 对这种情况做了优化：

- 当 `_goodid1 == _goodid2`（付的和收的是同一种商品），协议**完全跳过 AMM 计算**；
- 直接从小明账户扣 10 USDT → 直接打进小店账户；
- 只扣一笔固定的**执行费 `executeFee`**（约 0.05 USDT，极低）；
- 无滑点、无被夹风险、无定价不确定性。

对比：如果是 Uniswap，你要付 USDT 给商家，要么走 USDT → USDC → USDT 绕一圈（被两次滑点），要么走一个额外的托管合约。TTSWAP 的同币快路径**原生一步到位**。

6.4 两种交易方式的对比

场景	推荐函数	指定谁
普通 swap（我有 100，尽量多换）	`buyGood`	指定输入
精确收款（商家按 10 USDT 收款）	`payGood`（同币快路径）	指定输出
精确支付（我要恰好 10 ETH，付多少都行）	`payGood`（异币）	指定输出
订单系统（Web2 账单先生成再支付）	`payGood + external_info`	指定输出

7 商家与远程支付（X402）

7.1 问题：顾客自己下单，需要准备什么?

在普通的链上支付里，你作为顾客必须自己：

1. 准备好交易费（Gas，可能是 ETH）；
2. 连接钱包、签名、发送交易；
3. 等链上确认。

这对**从未接触加密货币的普通用户**非常不友好：他们没有 ETH、不懂 Gas、不会用钱包。

7.2 解决：X402 商家模式

TTSWAP 内置对 **X402 协议**的支持。思路非常简单：

> **顾客只签订单，不需要上链；"商家"代替顾客把订单送到链上，挣一笔固定执行费。**

角色：

- **顾客（Trader）**：用标准 EIP-712 在手机上签一个"我要买什么"的订单，不花 Gas；
- **商家（Executor）**：收到顾客的订单，验证签名,自己付 Gas 把订单送上链；
- **商场（TTSWAP）**：执行订单 + 从结果里**自动扣一笔固定执行费**给商家。

执行费叫 `executeFee`，合约常量 `50_000_000_000`（5×10^{10}）。

> **精确数字**：按 USDT（精度 6）折算 ≈ **0.05 USDT**（半分钱量级）。对顾客几乎为零，但积少成多足以让商家持续运营。

7.3 订单格式（EIP-712）

顾客签两种订单之一：

买单（对应 `buyGood`）：

```
buyGood(
    address _trader,      // 顾客地址
    address referral,     // 推荐人（可选）
    uint256 _goodid1,     // 付什么（T_GoodKey.toId()）
    uint256 _goodid2,     // 要什么
    uint256 _swapQuantity,// 付多少 / 底线拿多少
    bytes   data,         // 转账授权数据
    uint256 deadline,     // 过期时间戳
    uint256 nonce         // 防重放
)
```

付款单（对应 `payGood`）：

```
payGood(
    address _trader,
    address recipient,    // 收款地址
    uint256 _goodid1,
    uint256 _goodid2,
    uint256 _swapQuantity,
    uint256 external_info,// 业务元数据（订单号 + deadline）
    bytes   data,
    uint256 nonce
)
```

7.4 商家模式对商家的意义

场景：一个美团商家骑手拿着手机支付 TTSWAP 收款二维码，顾客只扫码签名，不需要 ETH、不需要 Gas 费。

- 骑手（Executor）挑起单就送进链上；
- 顾客（Trader）只签支付意图，不负担任何 Gas；
- 骑手挣：`executeFee`（约 0.05 USDT / 单）+ 聚合多单后的规模效应；
- 商家（Recipient）按 `_swapQuantity` 精确到账。

这让 **TTSWAP 具备 Web2 级"直接扫码支付"体验**。

商家只认收款币种，不关心顾客用什么币付款

传统链上收款有个隐形门槛：**商家得告诉顾客"请用 XX 币付款"**。顾客要是只有 ETH，商家只收 USDT，一笔支付就卡在了"找兑换入口 → 做 swap → 再付款"这条迂回路上。TTSWAP 的 `payGood` 从设计之初就把这件事收到了协议内部：

- **商家只做一件事**：在收款二维码 / 订单里填好自己**想收的币种 `_goodid2`** 与**精确到账金额 `_swapQuantity.amount1`**；
- **顾客用手上任何商品付款**：顾客在签单时把自己钱包里愿意花出去的**任一 ERC20（或原生 ETH）作为 `_goodid1`** 传进去即可——可以是 USDT、ETH、WBTC，也可以是任意已上架的长尾代币；
- **协议在一次调用内完成"自动兑换 + 精确到账"**：合约按 §6.2 的 exact-out 反推算出要花多少 `_goodid1`，一笔交易内在中央收银台里把顾客的币换成商家要的币，再把精确金额打到商家账户，过程对商家完全透明；
- **滑点保护仍在**：顾客在订单里设了 `amount0` 作为愿付上限，兑换率超过这个上限直接 revert，不会被"被动买单"。

商场场景直觉：商家把商品标价成"我要收 10 USDT"，顾客递过来的是"1 ETH 的会员卡"或者"某个长尾代币的余额"——收银台内部把会员卡兑成 USDT 再交给商家，商家始终只看见 10 USDT 到账。

对商家的三点好处：

1. **营收口径统一**：无论顾客用什么付款，账上流入的都是商家选定的收款币种（通常是稳定币 / 自家代币），财务口径清晰；
2. **覆盖所有持币顾客**：商家的潜在客户群 = 整个 TTSWAP 商场的持币用户，而不是只限"持有 USDT 的那一小撮"；
3. **无需自建兑换入口**：传统接入稳定币支付需要额外托管、swap 路由、滑点监控，TTSWAP 把这三件事合约原生做了。

特例：同币快路径（商家 = 顾客付款币种）

当顾客恰好手上就有商家要的那种币（ `_goodid1 == _goodid2` ），协议跳过 AMM，直接按 §6.3 的"同币快路径"走：

- 协议不做任何 swap，不收滑点；
- 只扣 `executeFee` （约 0.05 USDT）作为商家的中继费；
- 商家与顾客同样享受 Web2 级的"扫码即收款"体验。

7.5 小单实时签名 / 大单链上结算：面对面秒级交易

X402 打开了另一扇门：**同一张 EIP-712 签名，既可以是"现在立刻有效"的支付承诺，也可以是"晚些时候批量上链"的结算凭证**。商家可以按订单金额分档处理，复刻线下刷卡"免密小额 vs 密码大额"的分级风控思路——**日常面对面交易不再等 10–30 秒的链上确认**。

7.5.1 分档策略（商家自定阈值 T）

档位	金额范围	商家做法	顾客体验
小单	`≤ T` （例 ≤ 10 USDT）	本地验签名 → 立刻发货 / 提供服务；签名入**待结算队列**，晚些 / 下班前批量提交上链	扫码 → 1 秒放行，**不等区块确认**
大单	`> T`	把签名提交上链 → 等待确认成功 → 再发货	扫码 → 等 10–30 秒 → 放行

阈值 `T` 由商家自行设定，类比银行卡的"**闪付免密限额**"——同一家商户对不同类别商品甚至可以设不同 T，例如咖啡馆 T=20 USDT、电子产品 T=500 USDT。

7.5.2 为什么小单可以"先放行后上链"

对商家而言，**顾客签好的 EIP-712 订单本身就是一张可上链兑付的凭证**，类比一张有密码的支票：

- **签名是密码学收据**：包含顾客地址、付款金额、收款人、nonce、deadline 等字段，无法伪造或篡改；
- **上链时机由商家控制**：只要在 `deadline` 之前 + 顾客 nonce 未被使用掉、账户余额与授权还在，商家**任何时候**提交都能兑付；
- **失败了也能举证**：签名本身就是证据，可用于链下仲裁或 §11.3 董事会争议处理；
- **手续费更便宜**：批量提交（合并 `multicall` ）比一单一交易节省 gas。

7.5.3 风险边界与对冲手段

小单即时放行**确实存在小额违约风险**（顾客在商家上链前把资金转走或取消授权），但被三道措施收敛到极低：

1. **金额天花板**： `T` 就是商家单笔最大损失敞口，像银行卡的"免密额度"一样可控；
2. **`deadline` 超时自动作废**：顾客签的单子带过期时间，过期前商家必须上链，否则失效；
3. **顾客 `cancelNonce()` 可审计**：顾客如果频繁取消未上链签名，商家可直接把该地址移出"小单免密"名单（类似银行卡降额）。

对风险极度厌恶的商家，可以**始终把 T 设为 0**（所有订单都等上链），回退到默认行为；对追求极致速度的商家（便利店 / 咖啡馆 / 出租车），可以把 T 开到"日常客单价上限"，享受链下级体验。

7.5.4 一个典型门店场景

> **场景：街角咖啡馆，售价 3 USDT/杯。**
>
> 1. 顾客扫码，手机钱包弹出订单： `付 3 USDT 给 CoffeeShop` ；
> 2. 顾客按指纹签名 → **手机把签名发到商家 POS（BLE / NFC / 二维码）**；
> 3. POS 本地 `ecrecover` 验证 + 查签名字段是否与订单一致（**约 100 ms 完成**）；
> 4. POS 显示"✅ 支付成功" → 店员递咖啡 → 顾客离店；
> 5. 商家在**下班前**把当天所有签名打包 `multicall` 一次性上链， `executeFee` 仅扣一次、gas 摊薄到每单几乎为零。

整个客户流程 **≤ 2 秒**，与线下刷卡 / 扫码支付体验一致，**链的确认速度不再是面对面交易的瓶颈**。

7.5.5 小结：三个金额档 × 三个信任档

把 §6 / §7.3 / §7.4 / §7.5 的能力叠加起来，TTSWAP 商家支付可以覆盖从"咖啡小额"到"机构大额"的所有场景：

金额档	路径	延迟	适用场景
微小额 （ ≤ T_1 ，例 ≤ 10 USDT）	小单实时签名（§7.5）+ 同币快路径（§6.3）	**~1 秒**	便利店 / 咖啡 / 打车
中等额（ T_1 ~ T_2 ）	小单实时签名 + AMM 兑换（§7.4），	**~2-6秒， 链上执行成功时间**	餐饮 / 日用百货
大额 （ > T_2 ，例 > 1000 USDT）	大单链上结算	**链上确认时间**	电子产品 / 珠宝 / B2B 结算

商家通过调整 T_1 / T_2 两个阈值，就能在"极速体验"与"零违约风险"之间自由取舍——这是 TTSWAP 协议原生给到商家的**风控旋钮**。

7.6 为什么商家不会做恶?

你可能会担心：商家会不会帮你送上链时故意篡改订单？答案是**不能**：

- 所有订单都由顾客用 **EIP-712 签名**绑定订单内容；
- 合约在执行前**先 `ecrecover` 验证签名**，任何篡改都会让签名失效；
- 签名包含顾客递增的 `nonce` ，每个签名只能用一次；
- 可选 `deadline` 让签名自动过期；
- 顾客可以通过调用 `cancelNonce()` 主动作废所有**未上链**的签名。

7.7 与 Uniswap / 1inch Fusion 的对比

体验点	Uniswap	1inch Fusion	TTSWAP X402
顾客需要 Gas	✅ 要	❌ 不需要	❌ **不需要**
顾客需要钱包	✅ 要	✅ 要	✅ **要**（但纯签名）
支持任意商品 → 任意商品	⚠️ 需开 pair 小店	✅ 聚合	✅ **原生一店**
同币种精确收款	❌ 需绕路	❌ 需绕路	✅ **同币快路径原生**
中继费用	—	浮动	**固定 executeFee**

8 货架库存的"安全带"（safeLine）

商场最核心、但最容易被忽视的安全设计是：

> **每条货架的可交易深度，被 `safeLine` 上下界框在一个健康区间内。**

用两个 8 位配置字段（上界 / 下界）替代 v1 的 $Q \le I$ 硬约束。默认值为 0 表示"不额外收紧"；管理员可为长尾货架设更紧的区间。

8.1 为什么需要这条安全带

先设想一个**没有**安全带的商场：

> 有个骗子注册了一个"空气币" `$AIR` ，他告诉商场：
> "我给你们货架放 10,000,000 枚 AIR，初始价值$100,000 USDT$等价。"然后他开始卖‘
> AIR 给商场，拿走商场里真正的 `USDT / ETH`。 因为没有任何人真的想买 $AIR`，但商场还是乖乖给他 USDT / ETH。
> **结果**：商场里的 USDT / ETH 被空气币抽干，供货商全部被坑。

这是经典的"一般商品反向抽干"攻击，没有约束的话必然发生。

8.2 TTSWAP 怎么设这条安全带

每次大额 swap 分片计算结束后，合约检查：

- **卖入侧**（顾客把某商品"卖给"货架）：当前库存 Q 不得超过上界 → 否则 `TTSwapError(55)`；
- **买出侧**（顾客从货架买走商品）：当前库存 Q 不得低于下界 → 否则 `TTSwapError(56)`。

上下界由 `safeLine` 配置与基准库存 `I + 虚拟量` 按比例算出。管理员可通过 `modifyGoodByManager` 为不同货架定制松紧度。

8.3 这条安全带带来的三个行为规律

规律 1：大额卖压会在触及上界时被拒收，必须等买盘或撤货释放空间。

规律 2：新上架商品需要真实需求消化库存，才能继续承受卖单。

规律 3：外部价格暴跌时，砸盘无法一口气掏空货架——后续订单撞墙，套利与 LP 撤货逐步修复价格。

8.4 一般商品 vs 硬通货：两种待遇的本质区别

场景	一般商品	硬通货
大额卖压撞墙?	**会**（触及 safeLine 上界）	通常区间更宽，由管理员配置
单边补货计价	统一 `investGood` 池内价	同左
可作定价锚	可以但不权威	社区提权后更权威

规律：一般商品在"补货定价"和"卖出上限"两方面都更严格，这是为了保护供货商不被恶意商品稀释。硬通货的特权对应它经过"社区 II 类提案审批"的身份信用。

8.5 三重防御：抵御无常损失

除了"库存安全带"，TTSWAP 还有三层防御来保护供货商本金：

1. **第一道防线：协议级价值锚定（代码层）**
   供货凭证记录的是**历史进货数量**，撤货时按这个历史比例返还——**与当下价格无关**。即使货架价格剧烈波动，供货商拿回的商品数量是确定的。
2. **第二道防线：品牌方劣后缓冲（资金层）**
   硬通货 / 品牌方的初始流动性由发行方投入，**在极端行情下优先承担价值缺口**，相当于保险的"劣后级资金"。
3. **第三道防线：TTS 治理补偿基金（治理层）**
   极端情况下无法覆盖的损失，由 DAO 决议启动 TTS 拍卖 / 释放机制对受损用户补偿。

直觉：个人供货商不是独自承担 IL 风险，而是在三层防御下保证资金安全。通常手续费复利 + 补偿金足以将这个风险收敛到可接受水平。

9 供货商结账：怎么取回本金与利润

9.1 撤货的核心规则

供货商想撤货（取回本金 + 利润），调用 `disinvestProof(proofid, shares, gate, trader, signature)`：

1. 合约核对凭证归属（必须是你自己发起）；
2. **按凭证上的历史进货数量按比例返还**：
   - 你这次撤的份额占凭证总份额的比例 `X%`；
   - 按 `X%` 把**最初进货的数量**退给你；

- **与当前货架价无关**；

3. 计算累积的"利润"（当前货架净值 - 历史净值），按六方分账分发给供货商 / 品牌方 / 门店 / 推荐人 / 顾客 / 商场；
4. 撤货费（ `disinvestFee` ）按比例扣除，留在货架里（相当于给其他供货商提高单位份额净值，变相复利）；
5. 同步 TTS 解押：仅当该凭证曾记过 TTS 算力（ `isPromised` 货架）且本次撤资对应 `disinvestTTSValue > 0` 时，才 `unstake` 铸造 TTS。

9.2 为什么"按历史数量返还"能做到零无常损失

传统 AMM（Uniswap V2/V3）有个典型问题：**供货商撤货时，拿到的双币数量会因为中间价格变化而变形**。比如你存入 1 ETH + 2000 USDT（价值 4000 USDT），ETH 涨到 3000 时撤出，得到的可能是 0.8 ETH + 2400 USDT（价值 4800 USDT）。**价值涨了，但你的 ETH 数量少了——这就是无常损失。**

TTSWAP 的做法：

- 你存入的 10,000 枚 `$NEW` ，凭证上记录"历史进货 = 10,000"；
- 无论货架价格怎么波动，撤货时协议都按凭证返还 10,000 枚；
- 期间积累的手续费、福利注入等，作为**净值增量**额外给你。

结果：供货商撤货时得到的**商品数量不减反增**（本金 + 利润），这就是"零无常损失"的协议级实现。

9.3 撤货切片（Disinvest Chips）

巨鲸一次性撤太多货，会给其他供货商和顾客造成冲击。协议设了**撤货切片限额**：

- 每条货架都有一个配置项 `getDisinvestChips` ；
- 单次撤货的数量 / 价值不能超过切片上限；
- 超过就回滚 `TTSwapError(26)` 或 `TTSwapError(27)` 。

好处：平滑撤货对货架深度的冲击，防止巨鲸砸盘让其他供货商受牵连。

9.4 完整撤货示例

设 Bob 持有供货凭证：

- `shares = 10000` ， `invest = 10000` 枚 `$NEW` ；
- 期间货架累积了交易手续费；
- Bob 想撤 50%（5000 份）。

协议执行：

1. 按历史比例返还： `actualReturn = 10000 × 5000 / 10000 = 5000` 枚 `$NEW` ；
2. 计算当前净值对应的价值；
3. 净利润 = 当前净值 - 历史本金 ≈ 某个正数 Δ；
4. 扣撤货费 `disinvestFee` ；
5. 六方分账（Bob 通常能拿到 `liquidFee + customerFee` 两档再加本金）；
6. Bob 账户里：**5000 枚 `$NEW` + Δ 利润份额 + 退回的 TTS 算力**；
7. Bob 的凭证更新为剩余 `shares = 5000, invest = 5000` 。

9.5 撤货时的权限分发

`disinvestProof` 只允许 **proof 的所有者本人**调用：

- 协议通过 `S_ProofKey(trader, currentgood).toId() == proofid` 核对；
- 即使你有别人的签名也不能替他撤货（**没有签名代撤机制**）；
- 这是有意的设计——撤货涉及资金返还，风险面较大，禁止外人代操作。

9.6 供货同步挖 TTS：为商场做贡献的第二份收益

供货商在 TTSWAP 不仅赚交易手续费分佣，还**同步挖到 TTS 代币**。这条收益线对齐的是"**你为商场做出了多少贡献，商场就按价值分你多少TTS代币**"。

9.6.1 挖矿何时发生

动作	是否记 TTS 算力
`initGood` 开货架	**否**
`investGood` 补货	**仅当货架标记 `isPromised`**
`disinvestProof` 撤货	按撤出份额退回算力（有则 `unstake`）

每次符合条件的 `investGood`，合约在写回货架时自动执行 `L_Proof.stake(TTS, msg.sender, investvalue)` ——**同一笔交易内完成**，无需额外 `claim`。

9.6.2 TTS 算力按"贡献价值"记账

TTS 挖矿的计价口径与交易手续费完全解耦：

- **看价值，不看枚数**：补 1 万枚长尾币和 1 万 USDT，算力差很多——认的是 `investvalue`；
- **承诺型货架**：Launchpad / IDO 类项目由管理员设 `isPromised`，补货才计挖矿；
- **价值越大、算力越多**：硬通货补货通常效率最高。

9.6.3 每日增发池：2 亿总量 / 约 50 年线性释放

全部 LP 共享一个链上挖矿池。每 86,400 秒（1 天）协议自动结算一次：

$$\text{当日增发} = \frac{2\text{亿 TTS} - \text{当前流通量}}{18{,}250} \times \frac{\text{chainRatio}}{10{,}000}$$

（结算结果若低于 `1e12`，按 **1 TTS** 保底）

增发出来的 TTS 按所有活跃 LP 的**算力占比**分配。你算力占池子的 `X%`，当天就能分到 `X%` 的增发量。**没有"矿工费"、没有"补贴到期"，这条释放曲线内置在合约里**。

9.6.4 TTS 挖矿收益何时兑现

动作	TTS 算力	当下是否拿到 TTS 代币
补货 `investGood`（isPromised）	**算力立刻记账**	否（等日增发结算）
持续持有供货凭证	算力每天按比例参与分增发	否（收益在凭证上累积）
撤货 `disinvestProof`	按撤出份额**等比退回算力**	**是**，`L_Proof.unstake` 把对应的 TTS 收益铸到你账户

所以 §9.1 撤货第 5 步"同步把你在 TTS 代币池里对应份额的算力退回"，其实**还附带把你持仓期间累积的 TTS 收益一并到账**。撤多少份额就兑现多少算力对应的 TTS。

> 如果你只撤一部分，剩余份额继续挖矿；如果你全部撤出，算力清零，相当于"**退出矿池结算最后一批收益**"。

9.6.5 三份收益叠加：供货商的经济账

算上交易手续费分佣（§10）、零无常损失本金返还（§9.2）与 TTS 挖矿，TTSWAP 的供货商同时拿到三份回报：

收益来源	何时到账	计价货币
① 本金	撤货时按历史数量返还	你补的那种商品本身
② 交易手续费分佣（`liquidFee + customerFee`）	撤货时或 `collectCommission` 领取	对应货架的商品

收益来源	何时到账	计价货币
③ TTS 挖矿收益	撤货时 `unstake` 自动铸造	**TTS**

直觉：你为商场提供流动性的每一秒，都在同时收三种"利息"——现货本金的复利、交易分佣的现金流、以及代表商场所有权的 TTS 股权。前两者跟货架本身挂钩，第三者与整个商场的长期价值挂钩。

10 商场六方分账：每一笔交易被谁分了

10.1 六方分账：谁贡献,谁受益原则商场生态的经济基础

每一笔"撤货"或"交易"产生的利润都会被六方分账：

角色	在商场里的身份	典型占比	说明
供货商（LP）	给货架补货的人	**50% - 80%**	最大份额，奖励风险承担者
品牌方（Operator）	这件商品的发行方 / owner	1% - 3%	谁上架谁受益
门店（Gate）	展示这个商品的前端 / 聚合器	5% - 25%	谁引流谁受益
推荐人（Referral）	把顾客带进来的人	5% - 10%	拉新奖励
顾客（Customer）	顾客自己（通常用作"折扣返佣"）	5% - 10%	留存奖励
商场（Platform）	商场自己（社区金库）	2% - 8%	运营 + 回购销毁

10.2 这六方是 TTSWAP 的独创

对比传统 DEX：

协议	分账方	说明
Uniswap V2	100% LP	一方独享
Uniswap V3	LP + Protocol	两方
Curve	LP + veCRV holder	两方
TTSWAP	**6 方**	**协议级原生**

核心设计理念：TTSWAP 把"前端门店"、"推荐人"、"顾客折扣"**写进了协议合约**，让传统上要由中心化渠道 / 聚合器 / 营销团队去做的事情，直接由链上合约自动分发。

这让 TTSWAP 天然适合：

- **B2B2C 分销**：各家 SaaS 集成 TTSWAP 时，可以通过 Gate 角色拿链上返佣；
- **KOL 拉新**：Referral 关系链让社区推广合法合规；
- **用户忠诚度**：Customer 折扣自动回到用户本人，形成留存飞轮。

10.3 六方之和必须 = 100%

合约里有一条硬校验：

```
function checkGoodConfig(uint256 config) internal pure returns (bool) {
    // liquid + operator + gate + referal + cust + platform
    return (liquid + operator + gate + referal + cust + platform == 100);
}
```

每条货架的六方占比在 `goodConfig` 位里配置，**必须严格加起来 = 100%**。

这保证了每笔利润都会被完整分配，不会出现"漏账"或"黑箱抽成"。

10.4 分账路径的两种模式

模式 A：无推荐人（大多数冷启动场景）

- 供货商： `liquidFee + 本金返还`
- 门店： `operatorFee + customerFee`
- 商场： `profit − liquidFee + platformFee`

模式 B：有推荐人（典型的活跃用户）

- 品牌方： `operatorFee`
- 门店： `gateFee`
- 推荐人： `referFee`
- 供货商： `liquidFee + customerFee + 本金返还`
- 商场： `platformFee`

规律：推荐人一旦存在，品牌方 / 门店 / 推荐人 / 顾客 / 商场都能拿到对应份额；没有推荐人时，相关份额会归并到门店或商场。

10.5 佣金领取

所有佣金都累积在 `commission[角色地址]` 里：

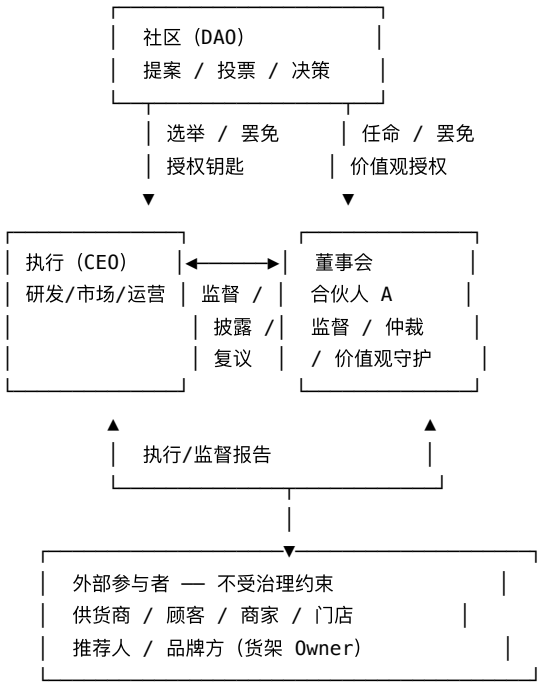
- 任何用户可以调用 `collectCommission(goodIds[], ...)` 批量提取；
- Market Admin 调用时还可以清空 `commission[address(0)]`（社区金库），供商场回购 TTS 销毁使用（由 CEO 按社区预算提案执行）；
- 合约里保留一个 1-unit sentinel（即使领空了也留 1），为的是避免"冷 SSTORE" 高 Gas 成本。

11 商场的决策架构与组织架构

TTSWAP 是一间**由代币持有人集体决策、由 CEO 带队执行、由董事会守护价值观**的去中心化商场。它不用传统上市公司那套"少数董事 + 多签 + 黑箱"的模型，而是**社区 – 执行 – 董事会的三角关系**，三方相互制衡、全链上透明：

- **社区（Community / DAO）**：TTS 代币持有人社区，拥有提案权、投票权，是最高决策层；
- **执行（Executive / CEO 团队）**：由 DAO 选举产生的 CEO 及其组阁的研发 / 市场 / 运营三个业务团队，持有合约 Admin / Manager 钥匙；
- **董事会（Board / 合伙人 A 类型持有者）**：由合伙人 A 类型 TTS 持有者组建，负责对运营进行持续监督、争议仲裁，以及对违反 TTSWAP 核心价值观的提案发起复议。

> **三角关系核心**：



这一章讲清楚三件事：

1. **决策架构（社区 / DAO）**：TTS 持币人如何提案 → 投票 → 执行；
2. **组织架构（执行 / CEO 团队）**：投票选出 CEO，CEO 如何组阁并推进业务；
3. **董事会（Board）**：合伙人 A 类型持有者如何监督运营、仲裁争议、守护 TTSWAP 核心价值观。

核心原则：

- **三权分立**：社区决策权、CEO 执行权、董事会监督权三方相互独立、相互制衡；
- **社区是顶层**：CEO 由社区选举，董事会由社区授权；CEO 和董事会**都对社区负责**；
- **董事会独立于 CEO**：CEO **无权任命或撤销**董事会成员；董事会不持合约钥匙，不参与日常业务；
- **品牌方（货架 owner）和外部参与者永远独立于所有治理层**，不受任何治理结果影响——任何用户随时可撤回自己的资金、领取自己的佣金；
- 合约底层的权限位（Admin / Manager）只是**社区决议的执行器**，跟着投票结果动态变更。

11.1 决策架构：TTS 持币人的 DAO

11.1.1 治理原则

- **人人可参与**：只要你持有 TTS，你就是商场的"股东"；
- **按量加权**：投票权重按 TTS 持有量；
- **链上透明**：所有提案、讨论、投票、执行记录都可链上追溯；
- **可问责**：每个通过的提案都对应一次或多次链上调用，谁发起、谁投票、谁执行都有证据。

11.1.2 谁可以提案?

> **持有一定数量 TTS 代币的地址** 可以发起提案。具体数值（例如流通量 1% / 固定 X 枚 / 快照阈值）由 DAO 在部署期间通过治理合约设定，并可在未来的提案中调整。

为什么设门槛：

- 防垃圾提案 —— 一个无门槛的治理会被"水军式提案"淹没；
- 让提案者有"皮肤在博弈中"（skin in the game）；

- 门槛保持在**一个活跃社区成员能够达到**的水平（不搞"只有 Whale 能提案"）。

11.1.3 谁可以投票？投票权怎么算？

- 任何持有 TTS 的地址**都可以投票**；
- 投票权重 = **你的 TTS 持有量**（具体是余额计票、还是锁仓加权、还是快照计票，由治理合约实现决定）；
- 不需要运行节点、不需要自己写代码，直接通过治理前端 / 多签 / 治理合约进行投票。

11.1.4 提案的生命周期

每个提案都会经历 5 个阶段：

```
阶段 1：起草（Draft）
  提案者在社区论坛 / Discord 把想法写清楚，征求反馈。
  目的：让提案内容成熟、预判反对意见。

阶段 2：正式提交（Submit）
  提案者在链上提交提案，此时冻结提案内容不可再改。
  需要满足提案门槛（持 TTS 数量）。

阶段 3：讨论 / 投票（Vote）
  进入固定投票期（例如 3–7 天，由治理合约定义）。
  所有 TTS 持有人按自己的票重投"赞成 / 反对 / 弃权"。

阶段 4：判定（Tally）
  投票期结束后：
    · 达到法定参与门槛（quorum） ✓
    · 赞成占比满足该类型提案要求 ✓
  两项都满足 → 提案通过。

阶段 5：执行（Execute）
  通过后，治理合约（或指定 executor）自动 / 手动调用对应
  的 Market / Token 合约函数，把决议落到链上。
  重大提案可能需要 Timelock 冷静期后才能生效。

（董事会会监督通过后的决策执行）
```

11.1.5 提案分类（四大类）

TTSWAP 的提案按"影响面"分成四大类，对应不同的通过门槛：

第 I 类：组织人事类

典型场景：

- 提名 / 罢免**商场 CEO**；
- 批准 / 否决 CEO 提名的职能团队负责人（研发、市场、运营的一把手）；
- 批准新增或裁撤职能团队；
- 特定情况下授权 / 撤销合约里 Admin / Manager 角色。

门槛建议：中等 / 高（因为直接决定执行团队的领导权）。

链上表现：通过后由治理合约调用 `setMarketAdmin` / `setTokenAdmin` 等任命 / 撤销合约角色。

第 II 类：协议变更类

典型场景：

- 调整商品的**默认费率**、**最大放大倍数 M**、**safeLine**；
- 提权一个商品为**硬通货（Value Good）**；

- 调整**六方分账占比上下限**；
- 合约升级到 （通过 `upgrade(newImpl)`）；
- 修改 `chainRatio` 等全局参数。

门槛建议：中等（需要兼顾灵活性和稳定性）。

链上表现：通过后治理合约调用 `modifyGoodConfig / modifyGoodByAdmin / setRatio / upgrade` 等函数。

第 III 类：预算与财务类

典型场景：

- 公售资金的提款用途（`withdrawPublicSell`）；
- 社区金库余额（来自 `commission[address(0)]`）的使用，例如：
  - 回购 TTS 并销毁的执行计划；
  - 营销预算拨款；
  - 开发者激励池；
  - 合作方返佣预算；
- 4C 分配池的新一轮份额分配（`addShare / permitShare` 批量空投）。

门槛建议：中高（涉及真金白银）。

链上表现：通过后治理合约调用 `withdrawPublicSell / collectCommission / addShare / permitShare`，资金流向链上透明可追溯。

第 IV 类：重大事项类

典型场景：

- **去中心化终局**（`disableUpgrade`）；
- 紧急**冻结整个商场 / TTS 合约**（`freezeMarket / freezeToken`）；
- 特殊情况下紧急**铸 / 销 TTS**（`mint / burn`，仅限合约迁移 / 漏洞修复）；
- 制定或修改**治理合约本身的参数**（提案门槛、投票期长度、quorum 等）。

门槛建议：高（例如超级多数 + 较长冷静期 / Timelock）。

链上表现：通过后执行无法回滚的链上操作，尤其 `disableUpgrade` 是一次性不可逆。

11.1.6 通过规则（三档）

提案类别	法定参与门槛（Quorum）	通过门槛	Timelock
I 组织人事	中	过半赞成	短（例如 24–48h）
II 协议变更	中	过半赞成	中（例如 48–72h）
III 预算财务	中高	过半赞成	中（例如 48–72h）
IV 重大事项	高	超级多数（≥ 2/3）	长（例如 7 天以上）

> **具体数值由治理合约部署时设定**，并可以通过一次 IV 类提案修改。这里只列分档原则。

11.1.7 提案通过后谁去"按那个按钮"

这是很多去中心化项目没讲清楚的细节：**提案通过了，谁来把链上调用做出去**?

TTSWAP 采用两种方式（由治理合约实现决定）：

- **方式 A：治理合约自动 executor**
  - 投票通过后，治理合约的 `execute(proposalId)` 可以由**任何人**调用（包括机器人）；
  - 函数内部检查提案状态、Timelock 是否过期，通过后自动调用目标合约函数；
  - 这种方式最去信任化，**谁按都一样，按钮背后的逻辑被代码固化**。

- **方式 B：CEO 作为 executor**
  - 提案通过后授权 CEO 或其团队执行（例如 CEO 多签）；
  - 适合需要**链下协作 + 链上多步组合**的复杂提案；
  - 过程可审计，但额外信任 CEO 团队不会拖延 / 扭曲执行。

核心：无论是哪种方式，**投票结果本身已经定案，执行只是"把钥匙插进锁里"**。

11.2 组织架构：CEO 组阁的执行层

11.2.1 商场 CEO（Market CEO）

身份定位：DAO 通过**组织人事类提案**选举产生的执行总负责人。对 DAO 负责，DAO 可以随时通过新提案罢免。

权限与职责：

- **组阁权**：提名研发 / 市场 / 运营三个职能团队的一把手（需要 DAO 通过或备案）；
- **调度权**：调度三个团队的资源、日程、优先级；
- **执行权**：把 DAO 的决议（通过的提案）推进落地；
- **对外代言**：代表商场对外协作（与交易所、审计所、合作协议、监管方等沟通）；
- **预算提议**：在 DAO 授权的预算范围内，向团队下拨资源（或发起更细的预算提案）；
- **紧急响应**：在遇到漏洞 / 攻击 / 市场异常时，第一时间组织人员响应，事后向 DAO 披露与追认。

合约层体现：

- CEO 持有 **`Market Admin` 多签钥匙**（商品核心配置 + Market 合约升级），这是 CEO 在链上的最高职权；
- CEO 同时持有 `Token Admin` 多签钥匙，用于 TTS 代币日常运营（公售 / 份额分配 / TTS 合约升级）；
- 这些钥匙**不是 CEO 本人的私产**，而是**社区授权的"工作钥匙"**；
- 社区随时可以通过 I 类提案撤销这些钥匙（调用 `setMarketAdmin(old_ceo, false)` / `setTokenAdmin(old_ceo, false)`）。

三件事情 CEO 不能独自做：

1. **不能独自按下 `disableUpgrade`** —— 去信任化是 **DAO Admin** 钥匙的职权，在**董事会**手里，不在 CEO 手里；必须由社区 IV 类提案通过后，由董事会执行；
2. **不能独自挪用社区金库** —— 预算使用必须走 III 类提案；
3. **不能独自调整自己的任命** —— 组织人事必须由社区投票决定。

11.2.2 研发团队（Tech Team）

由 CEO 提名一把手（Tech Lead）并组建团队。

主要职责：

- **智能合约开发与维护**：新功能、优化、漏洞修复；
- **审计协调**：与第三方审计所（OpenZeppelin / Trail of Bits / Spearbit 等）对接；
- **技术升级路线图**：规划 V2/V3 等新版合约；
- **SDK / 前端 / Subgraph** 等工具链维护；
- **紧急响应**：Bug bounty 对接、漏洞 patch、紧急熔断协作。

合约层体现：Tech Team 的一把手可能持有 `Market Manager` 的多签钥匙（用于日常运营微调，但合约参数不可超过 DAO 限定范围）；涉及**核心升级**（`upgrade`）需要 CEO 层授权。

11.2.3 市场团队（Market / BD / PR）

由 CEO 提名一把手（Market Lead）并组建团队。

主要职责：

- **品牌与公关**：商场的对外叙事、媒体关系、社区建设；
- **商户 BD**：拉新品牌方上架、对接支付 / 聚合器集成；
- **合作拓展**：与其他协议 / 基金 / 交易所 / 钱包的战略合作；
- **投资者关系**：与机构投资人 / VC / LP 等的对接；
- **内容运营**：白皮书、技术博客、教学视频、线下活动。

合约层体现：市场团队**通常不持有任何合约 Admin / Manager 钥匙**，更多是链下工作。涉及"提权新商品为硬通货"等事务，由市场团队发起讨论、提议 CEO 发起 II 类提案。

11.2.4 运营团队（Operations）

由 CEO 提名一把手（Ops Lead）并组建团队。

主要职责：

- **日常运营**：监控合约运行、Gas 成本、异常交易；
- **用户支持**：社区 / Discord / Telegram 的一线响应；
- **风控**：监测可疑地址、协同 `setBan` 封禁；
- **活动策划**：挖矿 / 空投 / 激励活动的执行；
- **社区金库日常操作**：按 DAO 预算提案的授权，定期执行 `collectCommission` / 回购 / 销毁；
- **数据披露**：定期向 DAO 披露运营数据、财务数据。

合约层体现：运营团队一把手可能持有 `Token Manager`（执行 `setBan`）或**执行者 / Executor** 节点（执行公售资金分配等常规任务）；敏感操作仍需 CEO 多签或 DAO 授权。

11.2.5 合约角色与组织岗位的映射

下表把"链上角色"与"组织岗位"对应起来，让读者清楚**钥匙在谁手里、授权来自哪里**：

链上角色	通常谁持有	授权来源	典型操作
DAO Admin	**董事会多签**	社区 I 类提案	永久锁定升级（`disableUpgrade`）、紧急铸销 TTS、任命其它 Admin
Market Admin	**CEO 多签**	社区 I 类提案	提权硬通货（`modifyGoodByAdmin`）、Market 合约升级
Token Admin	**CEO 多签**	社区 I 类提案	公售管理、份额分配、TTS 合约升级
Market Manager	研发 / 运营团队一把手	CEO（作为 Market Admin）授权	六方分账调整、紧急冻结货架 / 整店
Token Manager	运营团队一把手	CEO（作为 Token Admin）授权	封禁地址、紧急熔断 TTS
Stake Admin / Manager	运营团队一把手	社区 → CEO → 进一步授权	挖矿规则（预留）
Token Owner（品牌方）	开货架的用户本人	自然产生	调整费率 / power / chips

要点：

- 合约角色是**"工作钥匙"，**不是**个人的私产**；
- **DAO Admin 钥匙在董事会手里**，CEO **持有 Market Admin + Token Admin 钥匙**；
- 每把钥匙的分配都由社区 I 类提案决定；
- 每把钥匙可以被社区随时撤销（例如 `setMarketAdmin(old_ceo, false)` 、 `setDAOAdmin(old_member, false)`）；
- **Token Owner（品牌方）** 是例外——它跟"谁开了这条货架"绑定，自然产生，不由社区授权。

11.3 董事会（Board）—— 三角关系中的"监督 + 仲裁"一极

到目前为止，我们讲了两类角色：

- **社区（DAO）**：TTS 持币人通过提案和投票决定方向；
- **执行（CEO 团队）**：CEO 组阁三个业务团队负责落地。

如果只有这两层，仍有一个问题：**CEO 团队是否真的在按社区的决议执行？预算是否被合理使用？价值观是否被守住？**

为了回答这个问题，TTSWAP 在"社区 – 执行"二元结构之外，设立**第三极——董事会（Board）**，由**合伙人 A 类型 TTS 代币持有者**组建。与传统上市公司"董事会决策一切、雇佣 CEO"不同，TTSWAP 董事会**不直接决策、不执行业务**，只负责**监督运营、仲裁争议、守护 TTSWAP 的核心价值观**。

它与另外两方共同形成 **"社区 – 执行 – 董事会" 三角关系**：

方向	关系
社区 → 执行	选举 CEO / 授权合约钥匙 / 通过提案下达决议
社区 → 董事会	授权董事席位 / 通过提案调整董事会职能
执行 → 社区	汇报运营 / 按提案执行 / 接受罢免
执行 ↔ 董事会	接受监督 / 按要求披露数据 / 回应仲裁意见
董事会 → 社区	提交复议动议 / 提交监督报告 / 提出紧急召集

11.3.1 为什么需要"第三极"

- **监督执行**：CEO 团队有合约钥匙，负责日常推进，但需要有独立一方持续检查"他们是否按约定做事"；
- **仲裁争议**：当社区出现争议（提案解释分歧、品牌方纠纷、数据披露争议）时，需要一个超越利益的机构出具意见；
- **守护价值观**：TTSWAP 的核心价值观（零无常损失、一币一池、用户资金可取出）是协议的**底线**，不能因为短期利益而被动摇；
- **防止"多数暴政"**：单纯按持仓投票容易被巨鲸主导，需要一道**价值观过滤网**。

> **治理类比**：社区是立法机关、CEO 团队是行政机关、董事会相当于"审计院 + 宪法法庭"的综合体——**不执行具体业务，但对不符合宪法的行为亮黄牌**。
>
> 重要区别：TTSWAP 的"董事会"**不是西方公司法里"管理一切 + 雇佣 CEO 的董事会"**，而是 **"独立监察 + 价值观守护"** 角色，更接近审计委员会 / 监事会 + 宪法法院的综合体。

11.3.2 董事资格与产生方式

- **身份**：**合伙人 A 类型 TTS 代币持有者**；
- **资格细化**：具体的资格认定标准（例如 Partner 份额阈值、锁仓要求、行为评价等）由社区在后续的治理提案中确定。本白皮书把"合伙人 A 类型持有者"作为这个角色的通用名称，不硬编码具体阈值；
- **产生方式**：从符合资格的合伙人 A 持有者中选举产生，具体董事数量、任期、选举方式由社区 I 类提案定义；
- **独立性**：
  - **CEO 不能任命、不能撤销**任何董事；
  - 董事的任命 / 撤销 / 调整**必须经社区 I 类提案通过**；
  - 董事本人发生利益冲突时需**主动回避**相关议案。

11.3.3 董事会的五项核心职责

职责 1：持续运营监督

- 监督 CEO 团队**预算使用**的合理性（公售资金、社区金库、营销 / 研发 / 运营预算）；
- 监督**提案执行**是否符合原提案文本（是否按时、按量、按范围执行）；
- 审查**合约升级**的代码变更是否与社区提案描述一致；
- 在 `collectCommission(address(0))` 等涉及社区金库的操作前后，核对流水。

职责 2：争议仲裁

- 对**合约纠纷**（例如某次自动执行是否符合提案语义）出具仲裁意见；
- 对**品牌方纠纷**（例如某条货架是否存在恶意配置变更）出具仲裁意见；
- 对**社区纠纷**（例如预算分配、治理参数解释分歧）出具仲裁意见；
- 仲裁意见不是最终判决，而是**触发社区重新审议的依据**。

职责 3：价值观守护与复议权

- 对**任何**可能违反 TTSWAP 核心价值观的提案发起**复议动议**；
- "价值观"具体包括但不限于：
  - **零无常损失的 LP 返还机制**不得被削弱；
  - **一币一池 + Singleton 架构**不得被分叉切分；
  - **用户资金可取出的底线**（供货商可随时撤货、顾客可随时领取累积佣金）不得被阻断；
  - **六方分账总和 = 100%** 不得被突破；
  - `disableUpgrade` **一次性不可逆性**不得被任何方式绕过；
- 一旦董事会发起复议，**原提案进入冷静期**（例如额外 72h），社区需要**重新投票**：
  - 重新投票若以**超级多数**通过 → 原提案生效；
  - 否则 → 原提案作废。
- 董事会**不能单方面否决**提案，最终决定权仍在社区手里——"**黄牌 + 复议**"不是"红牌 + 否决"。

职责 4：透明度要求与定期披露

- 有权要求 CEO 团队按约定节奏披露：
  - **月报**：关键运营指标（TVL、交易量、手续费收入、活跃地址数等）；
  - **季报**：财务披露（公售资金余额、社区金库流水、各团队预算使用）；
  - **年报**：综合治理报告（提案通过率、紧急响应记录、价值观审查结论）。
- 披露内容**上链存证**（IPFS / Arweave + 链上 hash 引用），长期可追溯；
- 若 CEO 团队未按时披露，董事会可发起 I 类提案追责。

职责 5：紧急召集权

- 发现**重大异常**（疑似漏洞利用、CEO 团队集体失联、重大误操作）时，有权：
  - 请求 CEO 团队**紧急响应**；
  - 提议社区召开**加速投票**（缩短 quorum 达成时间）；
  - 在特殊情况下，动议"**临时冻结** CEO 团队的部分权限"（需社区快速投票批准）。
- 董事会**自身不持有合约钥匙**，无法直接调用 `freezeMarket`，但可通过提案让社区授权临时 executor 执行。

11.3.4 不做什么（边界清晰）

董事会的**权力边界**同样明确：

不做什么	解释
❌ 不参与具体业务执行	不组阁、不负责研发 / 市场 / 运营具体事务
❌ 不持有 Market Admin / Token Admin / Manager 钥匙	避免权责混同（只持 DAO Admin 钥匙）
❌ 不能单独做决定（除纯仲裁意见）	所有影响协议状态的操作都要走社区投票
❌ 不能单独罢免 CEO	罢免 CEO 必须走 I 类提案
❌ 不能单独分配预算	预算使用必须走 III 类提案

核心：董事会的力量是**"光照 + 黄牌 + 宪法级钥匙"**，不是"手术刀"。它用公开披露、仲裁意见、复议动议来影响决策；只有在社区决议授权后，才会动用 DAO Admin 钥匙执行"永久锁定 / 紧急铸销"等宪法级操作。

11.3.5 与社区 / CEO 团队的互动示例

互动场景 A：月度例行监督

```
月初：
  CEO 团队发布上月《运营月报》（TVL / 交易量 / 金库流水等）
  → 董事会审核
  → 抽查 1–2 笔重大操作的链上证据
  → 发布《监督简报》，上链存证
```

互动场景 B：预算争议

```
触发点：
  某次 III 类提案通过，预算用于"聘请外部顾问 5 万 USDT"
  → CEO 团队执行后，有社区成员质疑选聘流程
  → 董事会介入调查
  → 调查结论上链存证
    · 若合规 → 发布《合规确认书》
    · 若不合规 → 发布《仲裁意见书》 → 触发社区审议下一步
      （追回 / 追责 / 修改流程）
```

互动场景 C：价值观复议

```
触发点：
  社区通过某 II 类提案，拟修改"六方分账上限"，
  董事会认为新配置可能使 LP 份额被过度稀释
  → 董事会发起「价值观复议动议」
  → 原提案进入 72h 冷静期
  → 社区重新投票
    · 以超级多数通过 → 原提案生效
    · 否则 → 原提案作废
```

互动场景 D：紧急召集

```
触发点：
  董事会收到匿名线报：Market 合约存在严重漏洞
  → 董事会 1 小时内通过内部审议确认可信度
  → 公开呼吁 CEO 团队立即响应
  → 同时向社区发起《紧急投票提案》：授权 executor 暂时冻结受影响货架
  → 社区快速投票通过后，由 executor 执行 lockGood / freezeMarket
```

11.3.6 激励与问责

- **激励**：董事因其持续性工作（定期披露审核、仲裁意见撰写、复议动议处理）获得**社区金库中的固定津贴**（由社区预算提案批准）。津贴以 TTS 或 USDT 发放，上链存证；
- **问责**：
  - 若董事**未尽职**（例如漏审重大异常），社区可通过 I 类提案撤销该董事资格；
  - 若董事**滥用复议权**（例如频繁无理由发起复议拖延治理），社区可通过 I 类提案限制其复议权或撤销资格；
  - 若发生**严重失职 / 利益输送**，除撤销资格外，社区有权通过 IV 类提案发起惩罚性措施（例如销毁其锁定 TTS 份额）。
- **退出**：董事可随时主动退出；退出后不再享受津贴，但已完成工作的津贴照常发放。

11.3.7 合约层面的体现

董事会在合约里的直接对应是 **`DAO Admin 钥匙`**（ `UserConfig[255]` ）——即最高权限位。董事会通过多签持有 DAO Admin 钥匙，用于执行以下链上动作（全部由社区决议授权）：

- `disableUpgrade()`： **永久锁定合约升级**（一次性不可逆，去信任化终局）；
- `setDAOAdmin / setTokenAdmin / setMarketAdmin / setStakeAdmin`： 根据社区 I 类提案结果任命或撤销下一级 Admin；

- `mint(to, amount) / burn(from, amount)`：**紧急铸销 TTS**（合约迁移补偿 / 漏洞修复），仅在社区 IV 类提案通过后执行；
- `fixLeftShare()`：会计修正；
- `setEnv()`：部署期指定 Market 合约地址。

除了上述"宪法级"链上钥匙外，**董事会的日常监督 / 仲裁 / 复议职能目前主要以 "链下制度 + 链上事件存证" 方式运作**，与 Admin / Manager 的纯执行位图不同。

实施方式（分阶段逐步上链）：

1. **阶段 1（链下主导）**：董事会通过链下工具（Snapshot / 多签论坛）发布仲裁意见 / 披露简报，关键结论做链上 hash 存证；`DAO Admin` 钥匙仅在需要执行"永久锁定 / 紧急铸销"等硬决议时才使用；
2. **阶段 2（半链上）**：治理合约引入"复议锁"字段——董事会多签可在某 proposalId 上设置复议锁，触发 72h 冷静期；社区需要重新投票才能解锁；
3. **阶段 3（完全链上）**：董事会合约作为治理体系的独立模块上线，与主治理合约互联，`DAO Admin` 钥匙和"复议锁"实现**全自动"监督 + 仲裁 + 复议"**流程。

> **当前版本**：白皮书描述的是目标制度，董事会的具体合约化细节将作为**后续治理提案**由社区确认。

11.4 典型治理案例

案例 A：新商品 `$GEM` 晋升硬通货

```
事件背景：
  $GEM 上架 3 个月，TVL 100 万 USDT，日均交易量 ~30 万 USDT，
  社区讨论认为它可以晋升为硬通货。

流程：
  1. $GEM 品牌方 + 市场团队起草提案（"晋升 $GEM 为硬通货"）；
  2. 提案者提交 II 类提案，持 TTS 量达标；
  3. DAO 进入 72h 投票期；
  4. 投票通过（过半赞成 + 满足 quorum）；
  5. Timelock 冷静期（假设 48h）结束；
  6. 治理合约 executor 自动调用 modifyGoodByAdmin(GEM, 2**255)；
  7. 事件 e_modifyGoodConfig 被广播，$GEM 享受硬通货特权。

全程链上可审计，CEO 不直接参与批准，只协调社区讨论。
```

案例 B：漏洞紧急止损（链下协作 + 事后追认）

```
事件背景：
  白帽研究员通过 Immunefi 上报 Market 合约某分支存在漏洞，
  可能被利用抽走特定货架的流动性。

应急流程：
  1. 白帽上报 → 运营团队第一时间收到告警；
  2. 运营团队 15 分钟内完成复现，确认危险；
  3. CEO 召集研发 + 运营紧急会议；
  4. CEO 授权研发团队调用 freezeMarket()，整个商场暂停交易；
  5. 研发团队开发 V2 实现 + 提交审计；
  6. 审计通过后，CEO 发起 IV 类提案「升级到 V2 并恢复服务」；
  7. DAO 通过（可能走加速通道，缩短投票期）；
  8. 治理合约 executor 调用 upgrade(V2) → 恢复服务。
  9. CEO 向 DAO 提交完整事故报告，DAO 追认紧急处置。

设计精髓：紧急制动在"执行层"，长期修复必须经"决策层"同意。
```

案例 C：去信任化终局

```
事件背景：
  合约运行 2 年 + 3 次外部审计 + 0 次重大事故，社区讨论
  是否启动 disableUpgrade，完成最终去信任化。

流程：
  1. 社区长达数月讨论 + 外部审计专家意见征集；
  2. CEO 与研发团队完成最后一轮代码冻结；
  3. 提案者提交 IV 类提案「永久锁定合约升级能力」；
  4. DAO 进入 14 天投票期 + 7 天 Timelock；
  5. 超级多数（≥ 2/3）通过；
  6. 治理合约 executor 调用 disableUpgrade()；
  7. 自此 upgrade / freezeMarket / freezeToken 全部废止；
  8. 合约逻辑永久不可变。

此后 CEO 的职责仍在（外部协作 / 品牌运营），但**合约层面
的"钥匙"全部失效**，协议进入真正的"无老板"状态。
```

11.5 架构背后的制衡原则

去中心化商场不是"无人管"，而是"**没有人能单独说了算**"。TTSWAP 的三权分立架构靠四道制衡保证这一点：

制衡 1：决策权与执行权分离

- **决策权**归 DAO（TTS 持币人）；
- **执行权**归 CEO 团队（通过持有合约 Admin / Manager 钥匙）；
- CEO 不能独自发起组织人事 / 协议变更 / 预算 / 重大事项的决议——**这些必须走提案**；
- DAO 随时可以通过新提案罢免 CEO、撤销钥匙。

制衡 2：董事会（DAO Admin）独立于执行（Market/Token Admin）

- **董事会（持 DAO Admin 钥匙）**不持有 Market Admin / Token Admin / Manager 等业务钥匙；
- CEO（持 Market Admin + Token Admin）无权任命、无权撤销董事会成员；
- 对违反核心价值观的提案，董事会有"复议权"（黄牌 + 冷静期 + 社区重新投票），但**不能单方面否决**；
- 保证即使社区投票被短期利益裹挟，仍有一层"价值观过滤网"。

制衡 3：紧急响应与永久决议分离

- **紧急按钮**（`freezeMarket` / `freezeToken` / `lockGood`）在 CEO 团队手里，可毫秒级响应；
- **永久按钮**（`disableUpgrade` / 合约升级到全新版本）必须走 DAO 决议 + Timelock；
- 即使某个 CEO 团队成员一时冲动按下紧急按钮，最多**让服务暂停**，不会造成**不可逆后果**。

制衡 4：外部参与者不受治理约束

- 供货商、顾客、商家、推荐人、门店、品牌方**都不需要任何治理授权**；
- 即使 CEO 团队集体失职、社区提案系统瘫痪、董事会也出问题，普通用户依然可以：
  - 撤回自己的供货（`disinvestProof`）；
  - 买卖商品（`buyGood` / `payGood`）；
  - 领取累积佣金（`collectCommission`）。
- **这一层由合约硬编码，任何治理决议都无权阻断用户资金的取出**。

11.6 架构演进路线（从创始期到终局）

大多数去中心化协议都经历如下三个阶段，TTSWAP 也不例外：

阶段	决策架构	组织架构	董事会	标志性动作
阶段 1：创始期	创始团队多签 + 早期白名单持币人参与	创始团队兼任 CEO + 三个团队	链下主导，关键结论 hash 上链存证；DAO Admin 钥匙在创始团队多签手里	公售、首批硬通货提权、首批商品上架
阶段 2：稳定运营期	社区提案成为常态，CEO 由正式投票产生	CEO 组阁的三团队制度化	治理合约引入"复议锁"字段（半链上）；首届合伙人 A 董事会选举完成，接管 DAO Admin 多签	建立 Timelock、正式治理合约、董事会制度化
阶段 3：去信任化终局	`disableUpgrade` 触发；合约永久不变	CEO 成为"名义上"的角色，协议自运行	董事会合约作为独立模块上线，`disableUpgrade` 执行后 DAO Admin 钥匙永久失效	所有合约 Admin / Manager 钥匙失效

演进节奏：

- 阶段 1 → 阶段 2：通常需要 6–12 个月，标志是**有一个完整投票通过的 CEO 任命案**和**首届合伙人 A 董事会选举完成、接管 DAO Admin 多签**；
- 阶段 2 → 阶段 3：通常需要 2–3 年，标志是**至少 3 次外部审计 + 0 次重大事故 + 社区超级多数决议**；
- 阶段 3 之后：协议进入"**代码即宪法**"的终极状态，用户只依赖合约本身，不再依赖任何团队。

12 TTS 代币：商场最稀缺的"贡献凭证"

12.1 TTS 不是空气，是商场的虚拟股权等.

很多 DeFi 协议的治理代币本质上只是"投票凭证"，与协议实际使用脱钩。TTSWAP 的 TTS 代币不一样：

> **TTS 代币 = 这间商场的虚拟股权 + 投权票 + 市值驱动的解锁凭证。**

TTS 的几个核心属性：

- **名称 / 符号**：TTSwap Token / **TTS**；
- **精度**：12 位小数；
- **最大供应量**：**2 亿枚**；
- **通胀机制**：约 50 年线性释放，全部分给供货商（第 12.2 节）；
- **通缩机制**：商场手续费的"社区金库"部分 100% 回购 TTS 并销毁（第 12.4 节）。

12.2 供货即挖矿：TTS 自动到账

每次有人调用 `investGood` 往**承诺型**货架补货，协议把这批货的"价值"换算成 TTS 算力自动记账。

每 86,400 秒（1 天）协议自动结算一次增发：

```
每日增发量 = (2亿 − 当前流通量) / 18250 × chainRatio / 10000
(结果低于 1 TTS 时按 1 TTS 保底)
```

直觉：

- 剩余可发行量越少，每日增发越少；
- 增发量按"当前链的交易占比" `chainRatio` 分给不同的链；
- 约 50 年线性释放完毕；

- **只要供货商在货架上有货，TTS 就持续自动到账**——没有 "claim"、不需要额外操作。

12.3 4C 分配：创始股东怎么分初始 TTS

初始 4500 万枚 TTS（ `left_share` ）预留给四类贡献者，叫 **4C 模型**：

角色	占比	说明
创始人（Founder）	20%	项目发起人
合伙人（Partner）	12%	核心团队
价值贡献（Value）	44%	社区运营、技术开发
资本贡献（Capital）	24%	早期投资人、空投、公售

4C 设计理念：让"价值贡献（做真事的人）"占比最高，资本进来反而占比相对小。这是故意的——**劳动高于资本**。

12.4 翻倍解锁：市值不翻倍，代币不解锁

4C 的每个角色拿到的 TTS **全部锁定**，解锁必须满足一个苛刻条件：

> **只有商场里 TTS 相对于 USDT 的价格达到设定阈值，才能解锁一部分。每次解锁后，阈值翻倍。**

链上判定逻辑（简化）：

```
function shareMint() external {
    // 当前 TTS/USDT 价格必须高于 2^(已解锁次数) 倍起始价
    if (!market.ishigher(TTS, USDT, 2^metric × 2^128 + 20_000_000))
        revert;
    // 解锁量 = 剩余锁定量 / chips
    uint128 mintamount = leftamount / chips;
    shares[msg.sender].leftamount -= mintamount;
    shares[msg.sender].metric += 1;
    _mint(msg.sender, mintamount);
}
```

翻译成商场话：

> "我是一名创始股东，手上有 100 万枚 TTS，但全部锁定。
> 只有当 TTS 相对 USDT 的价格涨到超过第 N 次解锁的阈值，我才能解锁一份。
> 每次解锁，阈值翻倍。所以**如果价格从未上涨，我的 TTS 就永远锁住**。"

单次解锁上限 由 4C 角色的 `chips` 参数决定：

角色	占比	chips	单次解锁上限	触发示例
创始人	20%	20	1/20 = 5%	涨 1000× 累计解锁 ≈ 37%
合伙人	12%	14	1/14 ≈ 7.1%	涨 400× 累计解锁 ≈ 45%
价值贡献者	44%	12	1/12 ≈ 8.3%	涨 200× 累计解锁 ≈ 50%
资本贡献者	24%	8	1/8 = 12.5%	涨 100× 累计解锁 ≈ 60%

直觉：**创始人 与 合伙人 是项目的价值维护者，与商场长期绑定，因此 `chips` 最大、单次解锁比例最小（创始人 5%合伙人 ≈ 7.1%）——只有商场一轮又一轮跑出持续的长期价值，他们的代币才一档一档缓慢流出**，这保证了最了解协议的人不会在短期暴涨里提前下车。价值贡献者（Value）的解锁速度再快一档（ `chips` 更小，单次 ≈ 8.3%），鼓励"做事的人"早日变现。资本贡献者（Capital）的解锁门槛最低、单次上限最高（1/8 = 12.5%，涨 100× 累计 ≈ 60%），但总占比最少——资本出钱快回本快，但不占大盘子。

12.5 4C 翻倍解锁的三个经济含义

含义 1：价格不涨，代币永远不解锁

这是"**抗砸盘**"的终极设计：传统 VC 代币往往"上所即解锁"，带来巨大抛压。TTSWAP 把解锁与价格涨幅绑定——**TTS 价格涨上去了，代币才有机会变现**。

含义 2：代币释放节奏 = 市值增长节奏

解锁过程相当于一个**反馈循环**：

- 商场生意越好 → TTS 回购销毁越多 → TTS 价格越高；
- TTS 价格越高 → 解锁的代币越多 → 创始团队 / 投资人可以逐步变现。

这个反馈环让**代币释放自带"市场验证"节奏**，不需要 DAO 投票决议去"慢慢放水"。

含义 3：不需要持续治理投票

很多代币经济需要治理投票去解锁、去调节供给，容易陷入"公地悲剧"或大户操纵。TTSWAP 的翻倍解锁是**机制化治理——代码自己在执行**，不需要每年开 N 次治理投票。

12.6 公售：阶梯定价 + 硬顶

除 4C 分配外，TTSWAP 还保留一部分 TTS 用于**公开募集**：

阶段	已募资 USDT	兑换比例
Tier 1	≤ 87,500	1 USDT = **25 TTS**
Tier 2	87,500 – 162,500	1 USDT = **20 TTS**
Tier 3	162,500 – 250,000	1 USDT = **16 TTS**

- **硬顶 250,000 USDT**，超过直接回滚；
- 链上透明，谁都可以实时查到公售进度；
- 公售所得 USDT 由 Token Admin 通过 `withdrawPublicSell` 提取用于协议启动资金。

12.7 离线签名批量分配（`permitShare`）

如果需要**大规模空投 / 与合作方批量结算**，TTS 运营总监可以用 EIP-712 离线签一批分配单，受签方自己上链领取。

好处：

- **批量、异步、零 Gas** 签名；
- 适合空投、合作方配额、社区激励；
- 所有分配在链上留事件日志，完全可审计。

12.8 通胀 vs 通缩：双引擎

TTSWAP 的 TTS 经济是一个**双引擎发动机**：

引擎	方向	机制
通胀	发行	每日约 `(2亿 − totalSupply) / 18250 × chainRatio / 10000` 增发给 LP
通缩	销毁	商场 100% 社区金库利润在二级市场回购 TTS 并销毁

长期动态：

- 早期：供货商数量少，通胀 > 通缩 → 激励挖矿；

- 成长期：交易量放大，通胀 < 通缩 → 流通盘收敛；
- 成熟期：新品牌大量接入，TTS 稀缺性持续上升。

这个设计让 TTS 不会像大多数 DeFi 代币那样陷入"发行越多越贬值"的困境。

13 升级、熔断与永久不可变

TTSWAP 的合约采用**代理模式**（ `TTSwap_Market_Proxy` / `TTSwap_Token_Proxy` ）：

- **前台**（Proxy）：保存所有状态（供货凭证 / 货架库存 / 用户余额）；
- **后厨**（Implementation）：只保存业务逻辑；
- 前台通过 `delegatecall` 把请求转给后厨执行。

好处：**后厨可以换，前台不变，所有状态延续**。

三个关键操作：

操作	权限	说明
`upgrade(newImpl)`	**CEO**（持 Market Admin / Token Admin 钥匙）	把商场能力进行升级
`freezeMarket` / `freezeToken`	管理员 / TTS 运营员	紧急情况下把后厨设为 0，所有操作回滚（**紧急制动**）
`disableUpgrade`	**董事会**（持 DAO Admin 钥匙）	**永久锁定**，之后 `upgrade` 与 `freeze` 全部废止

两权分离：日常升级权（Market/Token Admin）在 CEO 手里（快速响应），永久锁定权（DAO Admin）在董事会手里（去信任化承诺）。

14 典型场景速查

场景	用什么函数	看哪一节
新商品上架（创业者）	`initGood`	§4
给现有货架补货（供货商）	`investGood`	§5
普通用户买东西（顾客）	`buyGood`	§6.1
精确付款（买家或商家）	`payGood`	§6.2
商家按 USDT 收款（零滑点）	`payGood` （同币快路径）	§6.3
代付 Gas / 无币用户下单	X402 签名模式	§7
供货商撤货 / 取利润	`disinvestProof`	§9
供货商 / 门店领取累积佣金	`collectCommission`	§10.5
品牌方调整费率 / 放大倍数	`modifyGoodByGoodOwner`	§11.2
批量办多件事	`multicall`	§6
提升商品为硬通货	`modifyGoodByAdmin` （CEO / Market Admin）	§11.2
紧急冻结某货架	`lockGood` （品牌方 or 管理员）	§11.3
TTS 解锁一部分	`shareMint` （需价格触发）	§12.4
公售参与	`publicSell`	§12.6

场景	用什么函数	看哪一节
签批量 TTS 空投	`permitShare`	§12.7

15 联系方式 & 法律许可

15.1 联系方式

- **官网**：https://ttswap.io
- **Twitter**：@ttswapfinance
- **Telegram**：@ttswapfinance
- **Discord**：ttswap
- **Email**：bussiness@ttswap.io
- **GitHub**：github.com/ttswap

15.2 法律许可（License）

文件类别	License	含义
核心合约（Market / Token / Proxy / L_Good / L_Proof）	**BUSL-1.1**	商业源使用许可，禁止在特定期限内商用 fork
接口与工具库（interfaces / L_Error / L_GoodConfig / L_TTSwapUINT256 …）	**MIT**	宽松开源，方便社区集成
ERC20 基础实现	**AGPL-3.0-only**	Copyleft 开源
EIP-1153 瞬态存储库	**GPL-3.0-or-later**	Copyleft 开源
WETH 接口	**GPL-2.0-or-later**	Copyleft 开源

> **商业用途声明**：如果你打算 fork TTSWAP 做商业产品，请先阅读 GitHub 仓库中的 `LICENSE` 文件，并联系 `bussiness@ttswap.io` 获取授权。

16 附录

附录 A：商场故事 ↔ 协议概念对照表

商场故事里的角色 / 物件	协议对应	代码 / 合约位置
整间商场	TTSWAP 协议	—
中央收银台	Singleton 合约	`TTSwap_Market_Proxy`
货架	每种商品的池	`mapping(goodid => S_GoodState)`
商品标识	`T_GoodKey`（类型+地址+子编号）	`toId()` → goodid
硬通货商品	Value Good	`modifyGoodByAdmin` 提权
一般商品	Normal Good	默认

商场故事里的角色 / 物件	协议对应	代码 / 合约位置
当前库存 Q	`currentState.amount1`	—
累计进货 I	`currentState.amount0`	—
市场价值 V	`investState.amount1`	—
供货凭证总份额 S	`investState.amount0`	—
供货凭证	Proof of Investment	`proofs[proofid]`
虚拟陈列深度	enpower	`getInvestPower()`
兑换曲线系数	固定为 2	`buyGoodInput/Output`
货架深度系数 M	`power`	`goodConfig` 用户区
安全线	safeLine	`TTSwapError(55/56)`
品牌方	Good Owner	`goods[goodid].owner`
管理员	Market Manager	`UserConfig[251]`
CEO	**Market Admin**	`UserConfig[252]`
TTS 运营员 / 总监	Token Manager / Token Admin	`UserConfig[253/254]`
董事会	**DAO Admin**	`UserConfig[255]`
商家	Executor	X402 / EIP-712 签名中继者
执行费	`executeFee`	合约常量 `5e10` （≈ 0.05 USDT）
供货商 / 补货	LP / `investGood`	—
顾客 / 下单	Trader / `buyGood` or `payGood`	—
同币种直接付款	`payGood` 同币快路径	`_goodid1 == _goodid2`
商家补贴	`goodWelfare`	—
商场TTS代币	TTS Token	`TTSwap_Token.sol`

附录 B：关键参数极限（合约硬约束）

参数	上 / 下限	含义
初始数量	$[500{,}000, 2^{109}]$	开新货架库存范围
初始自定价值	$[5 \times 10^{14}, 2^{109}]$	开新货架总价值范围
追投最小价值	$\geq 10^{12}$	单次 `investGood` 最小价值
最小交易价值	$\geq 10^{8}$	单次交易的最小转移价值
数量溢出保护	$< 2^{109}$	任何货架的数量字段溢出上限
执行费 `executeFee`	5×10^{10}（约 0.05 USDT）	固定常量
公售硬顶	2.5×10^{11}（250,000 USDT）	一次性不可突破
公售阶梯	25 / 20 / 16 TTS	每 USDT 换多少 TTS
TTS 总量	2×10^{20} wei = 2 亿 TTS	永久上限
日增发计算周期	86,400 秒	`_stakeFee` 每日触发一次

参数	上 / 下限	含义
佣金批量上限	≤ 100	`collectCommission` 单次最多领 100 个货架

附录 C：与主流 DeFi 的一张图对照

维度	Uniswap V2	Uniswap V4	Curve	Aave	TTSWAP
架构	每 pair 独立合约	Singleton + Hook	单池	借贷池	**Singleton 一币一池**
LP 是否双币	必须	必须	可单可双	单币	**单币**
无常损失保护	无	无	弱	N/A	**协议级 + 三重防御**
流动性聚合	pair 切分	部分	多池	N/A	**全协议聚合**
六方分佣	仅 LP	LP + 协议 + Hook	LP + veCRV	存款人 + 协议	**LP + 品牌方 + 门店 + 推荐人 + 顾客 + 商场**
原生 ETH	否	是	否	否	**是**
同币精确支付	否	否	否	否	**是（快路径）**
代付 Gas（无币用户）	否	部分	否	否	**原生 X402**
代币经济	治理	治理	veCRV（锁仓）	治理 + 安全模块	**供货即挖矿 + 4C 翻倍解锁 + 100% 回购销毁**

附录 D：术语总表

- **货架（池）**：每种商品在 TTSWAP 里的唯一池子，对应 `S_GoodState`；
- **供货凭证（Proof）**：每次补货产生的链上账本条目；
- **硬通货商品（Value Good）**：被社区提权为"定价锚"的商品；
- **一般商品（Normal Good）**：其它所有新商品，受 safeLine 约束；
- **CEO（Market Admin + Token Admin）**：由社区投票产生的执行最高负责人，持商品配置与合约升级钥匙；
- **管理员（Market Manager）**：日常运营 / 分佣 / 冻结货架的管理员；
- **董事会（DAO Admin）**：由合伙人 A 类型持有者组建，持 DAO Admin 钥匙，负责监督运营、仲裁争议、守护价值观，掌管 `disableUpgrade` / 紧急铸销；
- **商家（Executor）**：X402 签名中继者；
- **执行费（executeFee）**：`5e10`，约 0.05 USDT；
- **TTS**：TTSWAP 的治理 / 积分代币，总量 2 亿；
- **4C 模型**：Founder / Partner / Value / Capital 四类贡献者的分配模型；
- **翻倍解锁**：TTS 价格相对 USDT 每翻倍一次解锁一部分锁定代币；
- **六方分账**：LP / Operator / Gate / Referral / Customer / Platform 六方的手续费自动分发。

白皮书结束 · TTSWAP 去中心化商场 · 2026-07-07

> 有任何疑问或合作需求，请联系 bussiness@ttswap.io。
> 本文仅为协议说明，不构成任何投资建议；参与 DeFi 协议前请充分评估风险。